

Выбор NoSQL-решения: Архитектурный анализ моделей хранения и соответствующих сценариев применения

Ключ-Значение Системы: Принципы Хранения и Оптимизация Доступа

Ключ-значение системы представляют собой одну из самых простых и фундаментальных моделей хранения данных в экосистеме NoSQL ⁹. Их архитектура основана на принципе коллекции пар «ключ-значение», где каждый уникальный ключ однозначно идентифицирует свое связанное значение ^{17 31}. Эта модель эволюционировала как ответ на требования современных веб-приложений, которые требуют высокой скорости доступа к данным и масштабируемости, превосходя возможности традиционных реляционных баз данных в определенных сценариях ¹⁸. Основная концепция заключается в том, что база данных выступает в роли огромного словаря или хеш-таблицы, предоставляя минимальный набор операций: запись значения по ключу (PUT или SET), чтение значения по известному ключу (GET) и удаление пары по ключу (DELETE) ⁵³. Это сводит к минимуму накладные расходы на обработку запросов, поскольку отсутствует необходимость в парсинге сложных SQL-запросов, выполнении планов запросов или поддержке сложных механизмов транзакций между различными ключами ⁹.

Центральным элементом этой модели является то, что сама база данных не имеет представления о внутренней структуре значения ⁹. Значение рассматривается как неструктурированный байтовый поток или «черный ящик». Это означает, что система может хранить практически любой тип данных без необходимости его предварительного анализа или преобразования. Практически любые данные могут быть сохранены в виде значений, будь то примитивные типы, такие как строки и числа, сериализованные объекты в форматах JSON или XML, бинарные данные, например, изображения или файлы,

или даже более сложные структуры, такие как списки или словари, если они были предварительно сериализованы [14](#) [15](#). Такая абстракция делает ключ-значение системы чрезвычайно универсальными и гибкими, позволяя разработчикам хранить данные в том формате, который наиболее удобен для их приложения, не беспокоясь о соответствии жесткой схеме базы данных. Например, в некоторых системах, таких как MongoDB, используется формат BSON (Binary JSON), который является бинарным представлением JSON и позволяет хранить более широкий спектр типов данных, чем текстовый JSON [35](#) [61](#).

Хотя высокоуровневая модель проста, ее реализация на физическом уровне может быть значительно сложнее. Для достижения высокой производительности, особенно при записи больших объемов данных, многие современные ключ-значение хранилища, работающие с постоянным запоминающим устройством, используют продвинутую структуру данных под названием Log-Structured Merge Tree (LSM-Tree) [4](#) [57](#). LSM-Tree является популярной структурой для хранилищ ключ-значений, требующих высокой пропускной способности при записи [51](#). Этот подход решает проблему медленных случайных записей на диске. Вместо того чтобы записывать новые или обновленные данные в произвольные места на диске, что является медленной операцией, LSM-Tree собирает все изменения во внутренней структуре данных в памяти (часто это сбалансированное дерево или хеш-таблица). Одновременно каждая операция записи записывается в последовательный журнал на диске, известный как журнализация до внесения изменений (Write-Ahead Log, WAL) [42](#) [43](#). Этот журнал служит гарантией целостности данных; в случае сбоя сервера, содержимое WAL может быть использовано для восстановления данных, находящихся в памяти [43](#). Когда внутренняя структура в памяти достигает определенного размера, она "замораживается" и записывается на диск в виде готового к чтению файла, называемого SSTable (Sorted String Table). На диске могут существовать несколько таких SSTable'ов на разных уровнях иерархии. Периодически фоновый процесс слияния объединяет эти файлы, удаляя дубликаты и старые версии данных, и создает новые, более крупные SSTable'ы на следующем уровне иерархии [33](#). Этот механизм преобразует медленные случайные записи на диск в быстрые последовательные записи, что кардинально повышает скорость записи [33](#) [52](#). Таким образом, ключ-значение системы, использующие LSM-Tree, достигают очень высокой пропускной способности при записи, что делает их идеальными для write-heavy рабочих нагрузок [34](#). Однако эта оптимизация для

записи имеет свою цену: чтение может стать несколько сложнее. Чтобы найти ключ, система должна сначала проверить активную структуру в памяти, а затем, возможно, выполнить поиск по нескольким SSTable'ам на разных уровнях дерева на диске, что и приводит к некоторому усложнению операций чтения по сравнению с записью [34](#).

Такая простота модели хранения напрямую порождает специфические и хорошо определенные сценарии использования. Главным и наиболее очевидным применением ключ-значение систем является кэширование [46](#). Их способность выполнять миллионы операций получения за секунду делает их идеальным выбором для кэша верхнего уровня, который хранит результаты дорогостоящих вычислений, результаты запросов к другим базам данных или ответы API, чтобы избежать повторных обращений к источникам данных. Примером такого использования является Salesforce Platform Cache, который применяется для ускорения работы приложений [45](#) [47](#). Компании также используют KV-системы для кэширования ответов на запросы, что значительно снижает задержки для конечных пользователей [46](#). Анализ реального кэширования показывает, что такие системы сталкиваются с различными рабочими нагрузками, включая паттерны трафика, TTL (время жизни) и распределение популярности, что требует продуманных стратегий управления кэшем [50](#).

Другим важным сценарием является управление сессиями пользователей. Приложения, особенно веб- и мобильные, часто нуждаются в хранении временных данных, связанных с конкретным пользователем во время его сеанса работы. Эти данные могут включать информацию о входе в систему, предпочтения пользователя или состояние интерфейса. KV-система идеально подходит для этой цели, поскольку предоставляет быстрый способ связать сессионный идентификатор (ключ) с данными сессии (значением) и легко управлять сроком их жизни [46](#). Session Cache в Salesforce, например, представляет собой частное временное хранилище для каждого пользователя, которое можно использовать для повышения производительности [47](#). Управление конфигурациями — еще один распространенный use case. Приложения часто имеют множество параметров, которые могут меняться в зависимости от окружения (разработка, тестирование, производство). KV-хранилище может служить легковесным централизованным хранилищем для этих параметров, предоставляя приложению простой API для их получения [46](#).

Несмотря на кажущуюся простоту, ключ-значение системы играют критически важную роль в современных распределенных архитектурах. Они являются фундаментальным строительным блоком, на котором строятся более сложные системы, обеспечивая высокую производительность и масштабируемость для тех задач, где требуется максимальная скорость доступа по уникальному идентификатору, а сложная логика запросов вторична. Их успех заключается в радикальном упрощении модели данных, что позволяет избежать многих ограничений и накладных расходов, присущих реляционным базам данных, и сфокусироваться на одной главной задаче: быстром сохранении и получении данных по ключу.

Характеристика	Описание	Источник
Основная модель	Хранение данных в виде коллекции пар «ключ-значение»	9 17
Структура данных	Ключ — уникальный идентификатор. Значение — неструктурированный байтовый поток («черный ящик»)	9 31
Операции	Ограничены чтением (GET), записью (PUT/SET) и удалением (DELETE) по ключу	53
Индексирование	Нет индексирования по полям внутри значения	9
Физическая организация	Часто используют структуру данных Log-Structured Merge Tree (LSM-Tree) для оптимизации записи	4 57
Преимущества	Максимальная скорость доступа по ключу, простота, горизонтальная масштабируемость	9 10
Недостатки	Отсутствие сложных запросов, нет встроенного понимания структуры значения	9
Основные сценарии использования	Кэширование, управление сессиями пользователей, хранение конфигураций	46 47

Документные Базы Данных: Гибкость Структурированных Данных

Документные базы данных представляют собой развитие ключ-значение подхода, где «значение» уже имеет внутреннюю, понятную для базы данных структуру [13](#). Эта модель была создана для решения проблемы жестких схем, характерной для реляционных баз данных, предоставляя гибкость, необходимую для современных приложений, где данные могут быстро эволюционировать [10](#). Вместо того чтобы хранить значения в виде необработанных байтовых потоков, документные системы работают с полноценными документами, которые являются самоописывающимися единицами

данных [36](#). Наиболее распространенными форматами для хранения таких документов являются JSON (JavaScript Object Notation), BSON (Binary JSON), XML и YAML [14](#) [15](#). Эти форматы позволяют хранить данные в виде вложенных структур, таких как словари (объекты) внутри других словарей и массивы объектов, что делает их идеальными для представления сложных, иерархических данных, характерных для многих бизнес-объектов [15](#) [61](#).

Модель организации данных в документных базах данных часто описывается термином «схема без схемы» или «гибкая схема» [38](#) [48](#). Это не означает полного отсутствия схемы, а скорее указывает на то, что схема не является строго обязательной для всей коллекции документов. Каждый документ в коллекции может иметь свой собственный уникальный набор полей [28](#). Например, в коллекции продуктов один документ может описывать товар с характеристиками, отзывами и фотографиями, в то время как другой документ в той же коллекции может представлять услугу с совершенно другими полями, такими как продолжительность и стоимость часа. Хотя это дает огромную гибкость, на практике для обеспечения согласованности данных и упрощения программирования часто применяется определенная форма схемы, которая определяет общие ожидаемые поля для группы документов [38](#). Каждый документ в такой системе связан с уникальным ключом, обычно именуемым `_id`, который используется для его идентификации и быстрого доступа, аналогично ключу в KV-системе [38](#).

Ключевым архитектурным отличием документных баз данных от KV-систем является денормализация данных [27](#). Вместо того чтобы разбивать информацию о сущности на несколько связанных таблиц (как в реляционных базах данных), вся информация, относящаяся к одной сущности, хранится в одном документе [27](#). Например, описание продукта может включать не только его название и цену, но и массив отзывов, каждый из которых содержит имя автора, рейтинг и текст отзыва, а также массив изображений. Это значительно упрощает операции чтения: чтобы получить всю информацию о продукте, включая отзывы, приложению нужно выполнить всего один запрос к базе данных по ключу документа, вместо нескольких дорогостоящих операций объединения. Однако эта же денормализация может усложнить операции обновления. Если одно и то же значение встречается в нескольких местах (например, название категории продукта), его необходимо будет обновлять во всех документах, где оно используется, что может привести к избыточности данных и потенциальным проблемам с согласованностью.

В отличие от KV-систем, которые поддерживают только поиск по ключу, документные базы данных предлагают более богатый набор возможностей для индексирования и запросов. Они позволяют создавать индексы по полям внутри документов, что позволяет выполнять сложные запросы, а не только точный поиск по ключу [62](#). Разработчики могут запрашивать документы, удовлетворяющие определенным условиям по значениям полей (например, «найдите все продукты с ценой выше 100 долларов»), выполнять запросы по регулярным выражениям, а также осуществлять запросы к вложенным полям. Эта функциональность делает их значительно более мощными для приложений, которым требуется не просто хранить и извлекать данные, а также эффективно их фильтровать и агрегировать.

Гибкость и вложенная структура данных делают документные базы данных идеальными для широкого круга сценариев использования. Одним из классических примеров является управление контентом [63](#). Статьи в блогах, страницы веб-сайтов, новости — все это естественным образом представлено в виде структурированных документов, содержащих не только основной текст, но и метаданные (автор, дата публикации, теги), комментарии и изображения. Документная модель позволяет хранить всю эту информацию вместе, что упрощает как отображение контента на сайте, так и администрирование.

Управление профилями пользователей — еще одна область, где эта модель демонстрирует свою силу. Профили пользователей часто содержат большое количество разнородных и необязательных полей. Один пользователь может предоставить свой адрес доставки, а другой — нет. Один может указать свои хобби, а другой — предпочесть молчаливое поведение. Документная база данных с ее гибкой схемой позволяет хранить эти разнообразные профили без необходимости создавать огромную таблицу с множеством столбцов, большинство из которых будут содержать значения NULL для большинства записей [49](#).

Каталоги товаров — еще один яркий пример [63](#). Один документ может описывать товар, включая его характеристики, вариации (например, разные размеры и цвета), отзывы клиентов и изображения. Эта модель позволяет легко моделировать сложные, вложенные структуры продуктов, которые сложно представить в виде плоской реляционной таблицы. Блоги и социальные сети также являются хорошим применением: посты, комментарии и реакции (лайки) могут быть хранятся как связанные документы, где комментарии могут быть массивом вложенных документов внутри документа поста.

В целом, документные базы данных занимают важное место в экосистеме NoSQL, предлагая компромисс между простотой ключ-значения систем и мощностью реляционных баз данных. Они предоставляют достаточно структурированную среду для хранения сложных объектов, сохраняя при этом гибкость, необходимую для быстрой разработки и адаптации к меняющимся требованиям бизнеса.

Характеристика	Описание	Источник
Основная модель	Хранение данных в виде документов с гибкой схемой	13 36
Форматы данных	JSON, BSON, XML, YAML	14 15
Организация	Коллекции документов, где каждый документ имеет уникальный <code>_id</code> ключ	38
Схема	Гибкая схема ("схема без схемы"), где документы в коллекции могут иметь разные поля	38 48
Денормализация	Вся информация об одной сущности хранится в одном документе, что упрощает чтение	27
Индексирование	Поддержка индексирования по полям внутри документов для сложных запросов	62
Основные сценарии использования	Управление контентом, профили пользователей, каталоги товаров, блоги	49 63

Колоночные Базы Данных: Организация для Аналитики и Масштабируемости

Колоночные базы данных, также известные как базы данных с семействами колонок, представляют собой еще одно важное направление в мире NoSQL, ориентированное на решение задач, связанных с обработкой огромных объемов данных и высокой масштабируемостью [6](#). Их архитектура кардинально отличается от row-oriented моделей, принятых в традиционных реляционных базах данных, и даже от document-oriented подхода. Вместо того чтобы хранить всю информацию о одной записи подряд, они группируют данные по столбцам [2](#) [54](#). Это означает, что все значения первого столбца хранятся вместе, все значения второго столбца — вместе и так далее. Такая организация чрезвычайно эффективна для аналитических запросов, которые часто выполняют агрегацию по одним столбцам (например, `SUM(sales)`, `AVG(price)`), поскольку система должна прочитать данные только из одного

или нескольких столбцов, а не из целых строк, содержащих много ненужной информации ³.

Основной структурой организации данных в таких системах является таблица, которая состоит из множества колонных семейств ⁵⁵. Каждое семейство колонок — это контейнер для определенного типа данных, и все колонки внутри одного семейства обычно имеют схожую тематику. Самое главное, что семейства колонок могут быть определены со спарсой (разреженной) моделью данных ²⁸. Это означает, что каждая строка в таблице может иметь свой собственный уникальный набор колонок, и для строк, не имеющих значения для данной колонки, в базе данных не хранится ничего. Эта гибкость позволяет легко добавлять новые "атрибуты" для одной строки без необходимости изменять схему всей таблицы, что является значительным преимуществом по сравнению с реляционными базами данных, где добавление нового столбца может потребовать длительной миграции данных. Ячейка данных в такой системе однозначно идентифицируется комбинацией трех компонентов: **Row Key** (ключ строки), **Column Family** (семейство колонок) и **Column Qualifier** (качественник колонки) ²⁹. Качественник колонки идентифицирует конкретную колонку внутри семейства, а Row Key является основным идентификатором строки, аналогично первичному ключу в реляционной базе.

Эта архитектура делает колоночные базы данных незаменимыми для сценариев, требующих обработки больших объемов данных. Они широко используются в сфере больших данных и бизнес-аналитики, где компании собирают и анализируют огромные массивы информации для выявления трендов и принятия решений ⁷. Их способность выполнять быстрые агрегатные запросы по большим наборам данных является ключевым фактором их популярности. Еще одним важным применением является хранение данных сенсоров. Современные системы, такие как IoT, генерируют огромные потоки данных с переменными атрибутами, и колоночные базы данных обеспечивают превосходную производительность при записи этих данных и последующем анализе по временным рядам ²⁶. Аналогичная ситуация наблюдается в области логирования и мониторинга. Здесь возникает проблема высокой кардинальности данных, когда каждая запись (например, событие в системе) может иметь свой собственный набор полей. Колоночные базы данных эффективно справляются с таким разнородным потоком данных ⁸.

Физическое представление данных в колоночных базах данных часто использует ту же технологию, что и ключ-значение системы, — **Log-Structured**

Merge Tree (LSM-Tree) ¹ ³. Это позволяет им достигать очень высокой производительности при записи, что является критически важным для сценариев, таких как обработка потоков данных сенсоров или логирование ²⁶ ⁵⁵. Преобразование случайных записей в последовательные записи на диск, которое обеспечивает LSM-Tree, делает колоночные базы данных идеальными для рабочих нагрузок, ориентированных на запись ³⁴. Кроме того, многие системы этого типа, такие как Apache Cassandra, изначально проектировались как распределенные, отказоустойчивые системы, способные работать в десятках или сотнях серверов без единой точки отказа, что делает их подходящими для создания масштабируемых платформ ³⁰. В то же время, из-за своей оптимизации для агрегации по столбцам, они могут быть менее эффективны для запросов, которые требуют доступа к небольшому количеству полей из большого количества строк, так как для сбора данных приходится считывать данные из разных колонок, что может быть менее эффективно, чем чтение непрерывного блока строк в row-oriented системе.

Таким образом, колоночные базы данных являются специализированным инструментом, созданным для решения четко определенных задач. Они представляют собой компромисс между гибкостью и производительностью, предлагая уникальное сочетание высокой производительности при записи, эффективного хранения разреженных данных и быстрых аналитических запросов. Их выбор оправдан тогда, когда доминирующими требованиями являются обработка больших объемов неструктурированных или полуструктурированных данных, а аналитический анализ по отдельным атрибутам является основной целью использования системы.

Характеристика	Описание	Источник
Основная модель	Хранение данных в таблицах, где данные хранятся группами по столбцам, а не по строкам.	2 54
Организация	Таблицы состоят из семейств колонок. Каждое семейство может содержать различные колонки, и каждая строка может иметь разный набор колонок (разреженная модель).	7 28
Идентификация ячейки	Ячейка определяется комбинацией Row Key, Column Family и Column Qualifier.	29
Физическая организация	Часто используют структуру данных Log-Structured Merge Tree (LSM-Tree) для оптимизации записи.	1 3
Преимущества	Высокая производительность при записи, эффективное хранение разреженных данных, быстрая агрегация по столбцам.	26 55
Недостатки	Менее эффективны для точечных запросов по произвольным полям по сравнению с row-oriented системами.	Информация не доступна в предоставленных источниках
Основные сценарии использования	Big Data и аналитика, хранение данных сенсоров, логирование и мониторинг (observability).	8 26

Графовые Базы Данных: Моделирование Отношений как Центральная Парадигма

Графовые базы данных представляют собой радикальный сдвиг парадигмы хранения данных по сравнению со всеми остальными типами NoSQL и реляционными системами. Вместо того чтобы фокусироваться на самих данных (как в таблицах, документах или ключах), акцент делается на отношениях между этими данными [23](#). Эта модель идеально подходит для представления и анализа сложных сетей взаимосвязанных сущностей, где мощь системы раскрывается именно в способности быстро перемещаться по этим связям. Структура данных в графовой базе данных основана на трех фундаментальных понятиях: вершинах, ребрах и свойствах [24](#).

Вершины (также называемые узлами или сущностями) представляют собой объекты или концепции в вашей предметной области. Это могут быть люди, места, продукты, счета или любые другие сущности [24](#). Ребра (также называемые связями или ассерциями) представляют собой связи между вершинами. Они имеют направление и тип, который описывает природу связи. Например, между двумя вершинами «Человек» и «Компания» может существовать ребро типа «работает в» [24](#). Свойства — это атрибуты, которые

могут быть присвоены как вершинам, так и ребрам. Например, вершине «Человек» можно присвоить свойства «имя» и «возраст», а ребру «работает в» — свойство «дата начала работы» [22](#). Такая модель позволяет очень точно и интуитивно представлять реальный мир как сеть, где ключевым элементом являются связи.

Ключевым отличием нативных графовых баз данных от не-нативных является концепция индексной свободной смежности [64](#) [65](#). В большинстве баз данных, чтобы найти связанные данные, необходимо выполнить дорогостоящую операцию объединения или обратиться к внешнему индексу. В нативной графовой базе данных, таких как Neo4j, узлы хранятся таким образом, что каждый узел содержит прямые указатели (например, физические адреса в памяти или на диске) на свои смежные ребра [21](#). Когда система должна найти всех соседей определенного узла, она просто следует по этим внутренним указателям. Этот процесс происходит без обращения к внешнему индексу, что делает переход от одной сущности к другой чрезвычайно быстрым [64](#). Именно эта возможность обеспечивает нативным графовым базам данных преимущество в производительности при traversing графа (прохождении по связям), что является основной операцией в графовых алгоритмах [65](#).

Эта уникальная модель хранения делает графовые базы данных незаменимыми для задач, где анализ сетевых структур и отношений играет центральную роль. Одним из самых мощных и распространенных сценариев является системы рекомендаций [23](#). Представьте себе электронную коммерцию, где вы хотите предложить пользователю "продукты, которые также купили". Графовая база данных может моделировать покупателей, продукты и транзакции как граф. Поиск рекомендаций сводится к простому алгоритму: найти всех покупателей, купивших текущий продукт, а затем найти все продукты, которые купили эти покупатели, исключая текущий продукт. Такие запросы, включающие несколько шагов по связям, выполняются мгновенно.

Обнаружение мошенничества — еще одна область, где графовые базы данных демонстрируют свою исключительную силу. Мошеннические схемы часто вовлекают множество лиц, счетов и транзакций, связанных между собой сложными и неочевидными узлами. Графовая модель позволяет моделировать всех участников (люди, счета, IP-адреса) и все их действия (переводы, регистрации) как единую сеть. Алгоритмы анализа графа могут быстро выявлять аномальные паттерны, такие как сообщества, связанные через несколько промежуточных счетов, или необычные паттерны транзакций,

которые были бы невозможно обнаружить при анализе транзакций по отдельности [40](#) [41](#). Banking and Finance является ключевой отраслью для таких применений, где детекция кредитного мошенничества и отмывания денег является критически важной [41](#).

Знаний графы — это еще один важный сценарий использования [24](#). Знаний граф представляет собой карту знаний о предметной области, где факты моделируются как триплеты (субъект, предикат, объект), которые соответствуют графовым структурам (узел-ребро-узел). Например, "Альберт Эйнштейн -> родился в -> Ульм". Хранение этой информации в виде графа позволяет выполнять сложные вопросы и рассуждения, такие как "Какие ученые сотрудничали с Иваном Петровым и получили Нобелевскую премию?", что было бы крайне сложно и медленно сделать в традиционной базе данных.

Другие важные области применения включают анализ социальных сетей (идентификация лидеров мнений, исследование распространения информации), планирование маршрутов и транспортные сети (поиск кратчайшего пути), а также анализ зависимостей в ИТ-инфраструктуре (понимание того, как сбой в одном сервисе повлияет на другие).

В заключение, графовые базы данных предлагают уникальный подход к хранению данных, где отношения не являются дополнительной информацией, а составляют саму основу структуры. Их сила заключается в способности эффективно хранить и анализировать сложные, плотно связанные сети, отвечая на вопросы, связанные с тем, "как что связано с чем". Хотя они могут быть менее эффективны для простых CRUD-операций по сравнению с другими типами NoSQL, в своих нишах они являются непревзойденными инструментами для раскрытия скрытых паттернов и знаний, заложенных в данных.

Характеристика	Описание	Источник
Основная модель	Хранение данных как графа, состоящего из вершин (сущностей), ребер (связей) и свойств (атрибутов).	22 24
Парадигма хранения	Фокус на отношениях между данными, а не на самих данных.	23
Ключевая технология	Index-free adjacency в нативных графовых базах данных для быстрого обхода графа.	64 65
Преимущества	Экстремально быстрый обход графа, естественное моделирование сложных сетей, гибкость схемы.	23 64
Недостатки	Менее эффективны для простых операций CRUD по сравнению с другими типами баз данных.	Информация не доступна в предоставленных источниках
Основные сценарии использования	Системы рекомендаций, обнаружение мошенничества, знаний графы, анализ социальных сетей.	23 40 41

Справка

1. The use of a simplified LSM-tree architecture per column-family in... https://www.researchgate.net/figure/The-use-of-a-simplified-LSM-tree-architecture-per-column-family-in-wide-column-stores_fig2_324640550
2. Column-Family Databases - Meegle https://www.meegle.com/en_us/topics/nosql/column-family-databases
3. How LSM Tree improves write performance in NoSQL databases https://www.linkedin.com/posts/girish-s-guled-84711086_ever-wonder-how-wide-column-nosql-databases-activity-7299847004040359936-Jp0U
4. What is a LSM Tree? - DEV Community <https://dev.to/creativcoder/what-is-a-lsm-tree-3d75>
5. [PDF] PREPRINT - NoSQL databases: A survey - HAL <https://hal.science/hal-04660464/document>
6. Overview of Column Family Databases | PDF - Scribd <https://www.scribd.com/document/853954241/nosql-4>
7. Wide column vs column family vs columnar vs column oriented DB ... <https://stackoverflow.com/questions/63179561/wide-column-vs-column-family-vs-columnar-vs-column-oriented-db-definition>
8. The high-cardinality trap: why your observability platform is failing ... <https://clickhouse.com/resources/engineering/high-cardinality-slow-observability-challenge>

9. Choosing the right database: Tradeoffs and constraints - LinkedIn https://www.linkedin.com/posts/damini-bansal-93a606a4_systemdesign-architecture-engineeringdecisions-activity-7426839095303286784-1ZzP
10. Types of NoSQL Databases: Document, Key-Value, Column, and ... <https://quizlet.com/study-guides/types-of-nosql-databases-document-key-value-column-and-graph-b303fe3b-3c1b-4658-9483-5d8cb820ca62>
11. Write-Efficient LSM-Tree Storage via WAL-Time Key-Value Separation <https://arxiv.org/html/2506.04678v2>
12. [PDF] An Efficient Design and Implementation of LSM-Tree based Key ... <https://ceca.pku.edu.cn/media/lw/ae0a8602d9bd851889c303bd94d07a95.pdf>
13. Document-oriented Database - an overview | ScienceDirect Topics <https://www.sciencedirect.com/topics/computer-science/document-oriented-database>
14. What is a Document Database? Key Features, Use Cases, and ... https://www.linkedin.com/posts/maheshma_a-document-database-is-a-type-of-nosql-database-activity-7394909697209716738-7orT
15. Understanding NoSQL Database Types: Document <https://cacm.acm.org/blogcacm/understanding-nosql-database-types-document/>
16. Real-World NoSQL Use Cases - Meegle https://www.meegle.com/en_us/topics/nosql/real-world-nosql-use-cases
17. What is a key-value store? - Aerospike <https://aerospike.com/glossary/what-is-a-key-value-store/>
18. Key-Value NoSQL Examples - Meegle https://www.meegle.com/en_us/topics/nosql/key-value-nosql-examples
19. A Guide to Key-Value Databases - InfluxDB <https://www.influxdata.com/key-value-database/>
20. How to persist a graph data structure in a relational database? <https://stackoverflow.com/questions/18584598/how-to-persist-a-graph-data-structure-in-a-relational-database>
21. An overview of graph databases and their applications in the ... - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC8130509/>
22. [PDF] Survey: Graph Databases - arXiv <https://arxiv.org/pdf/2505.24758>
23. Understanding Graph Databases: A Comprehensive Tutorial ... - arXiv <https://arxiv.org/html/2411.09999v1>
24. What is graph analytics in knowledge graphs? - Milvus <https://milvus.io/ai-quick-reference/what-is-graph-analytics-in-knowledge-graphs>
25. (PDF) NoSQL Technologies in Big Data Ecosystems - ResearchGate <https://www.researchgate.net/publication/>

395674064_NoSQL_Technologies_in_Big_Data_Ecosystems_A_Comprehensive_Review_of_Architectural_Paradigms_and_Performance_Metrics

26. NoSQL Databases (PGDBD203) – VTU Sem 2 PGD Big Data Analytics <https://www.wonderslate.com/wpmain/aibook?siteName=books&bookId=109473>
27. [PDF] NoSQL databases: A survey - HAL <https://hal.science/hal-04660464v4/file/%28ARIMA%29%20Survey%20of%20NoSQL%20databases-final.pdf>
28. Column-family concept and data model - nosql - Stack Overflow <https://stackoverflow.com/questions/3245267/column-family-concept-and-data-model>
29. Data Model - Apache HBase <https://hbase.apache.org/docs/datamodel/>
30. Part 8 (Column Family Database - Cassandra As Example) - LinkedIn <https://www.linkedin.com/pulse/journey-database-world-part-8-column-family-cassandra-rasel-p5xwc>
31. Key-Value Stores Explained. Advantages & Use Cases. | Hazelcast <https://hazelcast.com/foundations/data-and-middleware-technologies/key-value-store/>
32. (PDF) BVLSM: Write-Efficient LSM-Tree Storage via WAL-Time Key ... https://www.researchgate.net/publication/392466722_BVLSM_Write-Efficient_LSM-Tree_Storage_via_WAL-Time_Key-Value_Separation
33. Single-Level KV Store with Persistent Memory - 知乎专栏 <https://zhuanlan.zhihu.com/p/64355248>
34. NoSQL Database Choices: Key-Value, Document, Column-Family ... https://www.linkedin.com/posts/thedevpiyush_okay-so-i-was-wrestling-with-database-choices-activity-7410945157644226560-7aCR
35. NoSQL Systems for Big Data Management - Academia.edu https://www.academia.edu/27773766/NoSQL_Systems_for_Big_Data_Management
36. Document-Based NoSQL - Meegle https://www.meegle.com/en_us/topics/nosql/document-based-nosql
37. Comparative Study of NoSQL Document, Column Store Databases ... https://www.researchgate.net/publication/274174394_Comparative_Study_of_NoSQL_Document_Column_Store_Databases_and_Evaluation_of_Cassandra
38. Data Modeling in MongoDB - Database Manual <https://www.mongodb.com/docs/manual/data-modeling/>
39. (PDF) NoSQL Databases and Data Modeling Techniques for a ... https://www.researchgate.net/publication/277890728_NoSQL_Databases_and_Data_Modeling_Techniques_for_a_Document-oriented_NoSQL_Database

40. Fraud Detection with Knowledge Graphs: A Protégé and VidyaAstra ... <https://dev.to/vishalmysore/fraud-detection-with-knowledge-graphs-a-protege-and-vidyaastra-approach-d0m>
41. Knowledge Graph For Fraud Detection - Meegle https://www.meegle.com/en_us/topics/knowledge-graphs/knowledge-graph-for-fraud-detection
42. Implement a KV database using the LSM Tree idea <https://segmentfault.com/a/1190000042102626/en>
43. The Engineering Behind Log-Structured Merge (LSM) Trees https://dev.to/agp_marka_62a62d1cdad70/distributed-database-internals-the-engineering-behind-log-structured-merge-lsm-trees-2258
44. Salesforce Misuse of Platform Cache Leads to Data Exposure <https://appomni.com/blog/salesforce-misuse-of-platform-cache-leads-to-widespread-data-exposure/>
45. Session Class | Apex Reference Guide - Salesforce Developers https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_class_cache_Session.htm
46. Chapter 14: KV and Hyperdrive | Architecting on Cloudflare <https://architectingoncloudflare.com/chapter-14>
47. How to Use Session Cache in Salesforce for Speed - LinkedIn https://www.linkedin.com/posts/rohit-chaudhary_salesforce-platformcache-sessioncache-activity-7319314172238807041-YJRc
48. ESDB: Processing Extremely Skewed Workloads in Real-time <https://dl.acm.org/doi/pdf/10.1145/3514221.3526051>
49. Document database use Cases - Amazon DocumentDB <https://docs.aws.amazon.com/documentdb/latest/devguide/document-database-use-cases.html>
50. A Large-scale Analysis of Hundreds of In-memory Key-value Cache ... <https://dl.acm.org/doi/fullHtml/10.1145/3468521>
51. An LSM Tree Augmented with B + Tree on Nonvolatile Memory <https://dl.acm.org/doi/full/10.1145/3633475>
52. An efficient design and implementation of LSM-tree based key-value ... https://www.researchgate.net/publication/266657104_An_efficient_design_and_implementation_of_LSM-tree_based_key-value_store_on_open-channel_SSD
53. Whole-system Persistence Made Efficient with Tree-structured ... <https://dl.acm.org/doi/10.1145/3742425>
54. Overview of NoSQL Database Types | PDF | No Sql - Scribd <https://www.scribd.com/document/702352041/Unit-4-DBP>

55. Understanding NoSQL: Use Cases, Design Principles, and Data ... <https://www.linkedin.com/pulse/understanding-nosql-use-cases-design-principles-data-mohammed-younis-3015f>
56. LearnedKV: Integrating LSM and Learned Index for Superior ... - arXiv <https://arxiv.org/html/2406.18892v2>
57. A Write-Optimized LSM Store for Cloud Block Storage with Robust SLA <https://dl.acm.org/doi/10.1145/3774424>
58. Integrating LSM and Learned Index for Superior Performance on SSD <https://arxiv.org/html/2406.18892v1>
59. NVLSM: A Persistent Memory Key-Value Store Using Log-Structured ... https://www.researchgate.net/publication/354250223_NVLSM_A_Persistent_Memory_Key-Value_Store_Using_Log-Structured_Merge_Tree_with_Accumulative_Compaction
60. [PDF] arXiv:2406.18892v2 [cs.DB] 11 Apr 2025 <https://arxiv.org/pdf/2406.18892>
61. NoSQL Databases - MongoDB - fxjwind - 博客园 <https://www.cnblogs.com/fxjwind/archive/2012/07/02/2573498.html>
62. Document Database - NoSQL | MongoDB <https://www.mongodb.com/resources/basics/databases/document-databases>
63. Types of NoSQL Databases - GeeksforGeeks <https://www.geeksforgeeks.org/dbms/types-of-nosql-databases/>
64. What is the difference between native graph storage and native ... <https://stackoverflow.com/questions/35611586/what-is-the-difference-between-native-graph-storage-and-native-graph-processing>
65. Neo4j Fundamentals-Native Graph Advantage - 红色MINI - 博客园 <https://www.cnblogs.com/z-cm/p/16220814.html>